MOSAIC 系统技术设计方案

> **MOSAIC** — Modular Orchestration System for Agent-driven Intelligent Control
> 面向 ROS 2 架构的 AI Agent 机器人任务调度系统

一、系统概述

MOSAIC 是面向人形机器人及带操作臂移动机器人的 AI Agent 任务调度系统。系统以大语言模型（LLM）为规划核心，以 3D 场景图为环境表征，通过模块化插拔架构实现感知-决策-执行的完全解耦。

设计重心在于 Agent 化智能调度框架本身，底层机器人能力（导航、操作、视觉等）使用 Nav2、MoveIt 2、SLAM Toolbox 等开源方案做 demo 级验证。

1.1 核心架构特征

- **3 层架构**：用户交互层 → AI Agent 核心层（含 AI Provider）→ 能力模块层 + ROS 2 基础设施
- **LLM-centric 规划**：任务规划由 LLM 基于 3D 场景图上下文进行常识推理和计划生成，SceneGraphValidator 验证计划可执行性
- **ARIA 多层记忆**：全局状态管理采用三层记忆架构——工作记忆、语义记忆、情景记忆
- **模块化能力插拔**：机器人能力按功能域组织为 CapabilityModule，支持运行时动态加载/卸载
- **视觉-操作闭环协作**：VisionModule 与 ManipulationModule 之间存在双向通讯

1.2 关键设计决策

决策	方案	理由
依赖倒置	Agent 核心仅依赖抽象接口	替换任何实现，核心零修改
AI Provider 位置	内置于 Agent 核心层	TaskParser、TaskPlanner、SceneGraphBuilder 均直接依赖，避免跨层调用
全局状态存储	ARIA 三层记忆 + EmbodiedRAG 检索	解决大规模环境下 LLM token 限制和注意力偏差

决策	方案	理由
场景图构建	SLAM 空间骨架 + VLM 语义填充	轻量级融合方案，适合 demo 阶段快速验证
ROS 2 封装	完全封装在 CapabilityModule 内部	Agent 核心不接触 ROS 2，保持框架通用性
任务执行	TaskExecutor = 优先级队列 + 执行调度	合并 TaskQueue，减少组件间通信开销
VLA 集成	VLA 作为 Capability 封装，与 MoveIt 2 方案并列可切换	任务级规划与端到端控制异步解耦

二、系统架构设计

2.1 系统分层总览

系统自上而下分为 3 层，每层只与相邻层交互，依赖方向始终向下（通过抽象接口）：

- **第 1 层 · 用户交互层**：CLI Interface，接收自然语言指令，展示执行结果
- **第 2 层 · AI Agent 核心层**：任务解析（TaskParser）、规划决策（TaskPlanner + SceneGraphValidator + CapabilityRegistry）、AI Provider（ModelProvider 接口 + LLMProvider/VLMClient 实现）、全局状态（RobotState + SceneGraph）、任务执行（TaskExecutor）、场景图构建（MapAnalyzer + SceneAnalyzer）
- **第 3 层 · 能力模块层 + ROS 2 基础设施**：导航模块、操作模块、视觉模块、搜索模块（SubAgent），以及 Nav2、SLAM Toolbox、MoveIt 2、传感器驱动等 ROS 2 底层组件

2.2 依赖倒置原则

Agent 核心（第 2 层）永远只依赖抽象接口，不依赖任何具体实现。替换任何具体实现（换 LLM 模型、换导航方案），Agent 核心代码零修改。AI Provider 作为核心层的内置组件，通过 ModelProvider 接口保持可替换性。ROS 2 被完全封装在 CapabilityModule 内部，Agent 核心不知道 ROS 2 的存在。

核心抽象接口包括：

- **ModelProvider**：AI 模型提供者抽象接口
- **Capability**：机器人能力抽象接口
- **CapabilityModule**：能力模块抽象接口

- **SceneGraphValidator**：场景图验证器抽象接口
- **SpatialQueryProvider**：空间查询抽象接口

2.3 数据流设计

以用户输入"导航到厨房"为例，展示完整数据流：

1. 用户通过 CLI 输入自然语言指令
2. TaskParser 调用 ModelProvider（LLM Function Calling）将指令解析为 TaskResult（intent=navigate_to, target=厨房）
3. TaskPlanner 调用 WorldStateManager.retrieve_context() 获取精简上下文（EmbodiedRAG 检索）
4. TaskPlanner 调用 LLM 生成候选计划，SceneGraphValidator 验证计划可执行性
5. 验证通过后生成 ExecutionPlan，交由 TaskExecutor 执行
6. TaskExecutor 调用对应 Capability.execute()，Capability 内部封装 ROS 2 通信
7. 执行结果沿管道回传至 CLI 展示

2.4 执行中反馈驱动决策流

当执行过程中遇到异常（如路径阻塞），系统通过以下流程实现 LLM 迭代重规划：

1. Capability 通过 feedback_callback 上报 FeedbackEvent（如 BLOCKED）
2. TaskExecutor 将事件转发给 WorldStateManager，更新场景图状态
3. WorldStateManager 通知 TaskPlanner 状态变化
4. TaskPlanner 调用 retrieve_context() 获取更新后的上下文，请求 LLM 重规划
5. 新计划经 SceneGraphValidator 验证后替换当前计划继续执行

三、核心组件设计

3.1 抽象接口层（interfaces_abstract）

定义系统所有核心抽象接口，是整个架构的契约层。

ModelProvider：AI 模型提供者抽象接口，定义 `parse_task(context) -> TaskResult` 和 `get_supported_intents() -> list[str]` 两个核心方法。

Capability：机器人能力抽象接口。纯执行 Capability 只依赖 ROS 2 Adapter；SubAgent Capability 额外依赖 ModelProvider + WorldStateManager（只读）。核心方法包括

`execute(task, feedback_callback)` 、 `cancel()` 、 `get_status()` 、 `get_capability_description()` 。通过 `is_sub_agent()` 标识是否为 SubAgent 类型。

CapabilityRegistry：能力注册中心，管理 CapabilityModule 和 Capability 的注册、注销和意图解析。支持模块级批量注册/注销。

SpatialQueryProvider：ARIA 对外暴露的空间数据访问协议，CapabilityModule 通过此接口获取空间信息，不直接依赖 SceneGraph 数据结构。核心方法包括 `resolve_location()` 、 `get_navigable_targets()` 、 `get_room_topology()` 、 `is_path_clear()` 、 `get_objects_in_region()` 。

3.2 Agent 核心层（agent_core）

架构核心原则：LLM 规划 + 场景图 Grounding + 形式化验证。

3.2.1 TaskParser（任务解析器）

管道入口，负责将自然语言输入转化为结构化的 TaskResult。自身不做 NLP 处理，而是委托给 ModelProvider。接收 CLI 传入的 TaskContext，调用 ModelProvider.parse_task() 获取结构化解析结果，校验合法性后传递给 TaskPlanner。

3.2.2 WorldStateManager / ARIA（多层记忆架构）

> **ARIA** — Agent with Retrieval-augmented Intelligence Architecture

参考 EmbodiedRAG（JHU APL, 2024）的 3D 场景图向量化检索、ReMEmbR（NVIDIA, 2024）的长时语义记忆、GraphRAG 的知识图谱混合检索，设计三层记忆架构。

三层记忆模型：

记忆层	类比	存储内容	存储方式	生命周期
工作记忆 WorkingMemory	人类短期记忆	RobotState（位姿、传感器、能力状态）	内存数据结构	实时覆写
语义记忆 SemanticMemory	人类长期知识	SceneGraph 节点/边 + 向量嵌入索引	图结构 + VectorStore	持久化，增量更新
情景记忆 EpisodicMemory	人类经验回忆	任务执行历史（成功/失败经验）	向量嵌入 + 时间戳索引	持久化，按时间衰减

核心目标：LLM 规划时不再接收全量场景图，而是通过任务驱动检索获取精简、相关的上下文，解决大规模环境下的 token 限制和注意力偏差问题。

EmbodiedRAG 检索流程（参考 arXiv:2410.23968）：

1. **Pre-Retrieval**：LLM 基于任务文本推断相关实体（无需环境知识）
2. **向量检索**：在 VectorStore 中检索 top-k 相似节点
3. **子图 Grounding**：以检索节点为锚点提取诱导子图 + 属性过滤
4. **Self-Query**：LLM 规划过程中的 thoughts 反馈给检索器，动态扩展检索范围
5. **经验召回**：从 EpisodicMemory 检索相似任务的历史经验
6. **上下文组装**：精简子图 + RobotState + 历史经验 → PlanningContext

实测可将 token 消耗降低一个数量级（EmbodiedRAG 论文数据：90% 累计 token 减少），规划时间减少 70%。

与 SayPlan 语义搜索的对比：

维度	SayPlan（折叠/展开）	EmbodiedRAG（向量检索）
检索方式	LLM 手动 expand/contract 节点	向量相似度自动检索 + 子图 Grounding
LLM 负担	LLM 需要理解图结构并操作	LLM 只需描述需求，检索器自动完成
动态适应	需要预构建完整场景图	支持在线构建、增量索引
反馈机制	无	Self-Query 机制动态扩展检索范围
经验利用	无	情景记忆召回历史经验辅助规划

本系统采用 EmbodiedRAG 方案替代 SayPlan 的折叠/展开机制，同时保留 SayPlan 的层级化场景图表征作为底层数据结构。

3.2.3 ARIA 空间数据适配层

ARIA 内部数据结构与具体感知方案之间通过 SpatialQueryProvider 抽象接口解耦。不同感知阶段通过实现不同的 Provider 适配器来桥接：

阶段	感知方案	Provider 实现	数据来源
Step 1	2D SLAM + YAML 标记点	YamlMapProvider	locations.yaml + 栅格地图
Step 2	2D SLAM + 房间分割	Slam2DProvider	MapAnalyzer 输出 + 标记点
Step 3	完整 3D 场景图	SceneGraphProvider	SceneGraph 层级查询

导航模块不关心 ARIA 内部是 YAML 文件、2D 栅格地图还是 3D 场景图——它只通过 `resolve_location("厨房")` 获取坐标。感知方案升级时，只需替换 SpatialQueryProvider 的实现，所有消费者零修改。

3.2.4 TaskPlanner（LLM 规划决策中枢）

系统的"大脑"，基于 LLM 常识推理 + 3D 场景图 Grounding 做出所有规划决策。持续监控 WorldState，在状态变化时自动评估并触发 LLM 重规划。

核心职责：

1. **初始规划**：TaskResult → retrieve_context() → LLM 生成计划 → Validator 验证
2. **持续监控**：WorldState 变化 → 评估当前计划有效性 → 必要时重规划
3. **执行结果处理**：成功 → 更新场景图；失败 → LLM 重规划
4. **迭代验证**：计划经 SceneGraphValidator 验证，不通过则携带反馈重新生成
5. **Self-Query 反馈**：规划 thoughts 反馈给 WorldStateManager 扩展检索范围

Planner 决策矩阵：

触发事件	行为	LLM 推理过程
初始用户指令	生成 ExecutionPlan	retrieve_context() → LLM 生成计划 → Validator 验证
动作执行成功	更新场景图，推进计划	更新 SceneGraph + 情景记忆
动作执行失败	LLM 重规划	失败原因 + retrieve_context() → LLM 生成替代方案
场景图变化（路径阻塞）	评估 + 可能重规划	LLM 基于更新后检索子图判断计划是否仍可行
新高优先级任务	抢占 + 重规划	LLM 综合新旧目标 + 当前状态生成新计划

LLM 规划优于硬编码规则的原因：异常处理从"程序员预设规则"变成了"LLM 基于场景图上下文的常识推理"，泛化能力质的飞跃。路径阻塞时 LLM 基于常识推理出绕路方案，新物体出现时 LLM 自然理解其可交互方式，无需预定义所有可能的规则。

3.2.5 TaskExecutor（执行器）

内置优先级队列，负责调度和执行。核心原则：**不做决策**，决策交给 Planner。

- 接收 ExecutionPlan，按优先级入队
- 按序取出 PlannedAction，调用 Capability.execute()
- 将结果和反馈上报给 Planner 和 WorldStateManager
- 反馈事件更新 WorldState（Planner 自动感知变化）

3.2.6 SceneGraphBuilder（3D 场景图构建器）

参考 SayPlan（CoRL 2023）的 3D Scene Graph 方案，使 Agent 从"已知环境内规划"进化为"面向新环境的自主探索与理解"。

核心方案：SLAM 空间骨架 + VLM 语义填充

融合流程：

1. SLAM 建图阶段：SLAM Toolbox 构建栅格地图，提取空间拓扑
2. 房间分割：对栅格地图做连通区域分析 + 窄通道检测，自动分割出 Room 节点
3. 语义巡回：机器人在每个 Room 的关键位置拍照
4. VLM 分析：每帧 RGB 送入 VLM，识别物体、表面、容器及空间关系
5. LLM 归纳：汇总多帧 VLM 结果 + 房间拓扑，归纳语义属性
6. 场景图组装：SLAM 拓扑 + VLM 物体 → 完整层级场景图（Building → Floor → Room → Object）

三层技术架构：

层次	职责	技术方案
空间感知层	建图 + 房间分割 + 拓扑提取	SLAM Toolbox + 栅格地图连通区域分析
语义感知层	物体/关系识别 + 房间语义标注	VLM（GPT-4V / 开源 VLM）+ RGB 相机
知识融合层	空间拓扑 + 语义信息 → 场景图	LLM 归纳 + SceneGraphValidator

子组件：

- **MapAnalyzer**：栅格地图空间分析，连通区域分析 + 窄通道检测 → 房间分割 + 拓扑提取
- **SceneAnalyzer**：VLM 语义分析，从 RGB 图像中提取物体和空间关系
- **SceneGraphValidator**：验证 LLM 生成的计划在场景图约束下是否可执行，包括空间约束验证和状态约束验证

场景图构建的两种模式：

- **全量构建**：机器人首次进入新环境，自主探索 → 全量场景分析
- **增量更新**：运行时发现新物体/状态变化，局部分析 → 增量合并

四、能力模块设计

4.1 模块化插拔架构

机器人能力按功能域组织为独立的 CapabilityModule，每个模块内部自治，对外通过标准 Capability 接口暴露。

Capability 分为五类：

类型	特征	示例
纯执行	直接调用 ROS 2，不需要 LLM	Navigation、Mapping、Motion
感知	调用视觉/传感器获取环境信息	ObjectDetection、PoseEstimation
协作	需要跨模块通讯	Grasp（依赖 VisionModule）
VLA	内部运行端到端 VLA 模型高频闭环控制	VLAManipulation
SubAgent	内部需要 LLM 推理来完成子任务	ObjectSearch

4.2 导航模块（NavigationModule）

- **NavigationCapability**：封装 Nav2 NavigateToPose Action，支持 navigate_to、patrol 意图
- **MappingCapability**：封装 SLAM Toolbox Service，支持 start_mapping、save_map、stop_mapping
- **MotionCapability**：通过 ROS 2 Topic 发布运动指令，支持 rotate、stop
- **LocationService**：模块内共享服务，维护语义地名到坐标的 YAML 映射，支持热加载

4.3 操作模块（ManipulationModule）

封装操作臂控制能力，内部通过 MoveIt 2 进行运动规划，依赖 VisionModule 提供目标检测和位姿估计。

- **GraspCapability**：执行前请求 VisionModule 进行目标位姿估计，基于估计结果规划抓取轨迹
- **PlaceCapability**：规划放置轨迹并执行，完成后请求 VisionModule 确认放置结果
- **ArmControlCapability**：底层关节级控制（MoveIt 2 封装）

视觉-操作协作通讯通过模块间回调接口实现（非 ROS 2 Topic），保持模块间松耦合。

4.4 VLA 端到端操作能力（演进预留）

ManipulationModule 预留 VLA（Vision-Language-Action）端到端操作能力接入，与 MoveIt 2 方案并列，通过配置切换。

VLA 控制架构：VLAManipulationCapability 接收语言指令，启动高频控制循环（50-200Hz），VLA 模型端到端推理生成关节动作流，通过 ROS 2 关节控制器执行。对外仍是标准 execute() 接口，内部高频循环对上层不可见。

关键设计要点：

- 时间尺度分离：MOSAIC 规划层（秒级）与 VLA 控制层（毫秒级）异步解耦
- 模型接入：通过 LeRobot/OpenPI 等开源框架加载模型权重
- 配置切换：agent_config.yaml 中选择 "moveit2" 或 "vla"，Agent 核心零修改

MoveIt 2 与 VLA 方案对比：

维度	MoveIt 2	VLA
控制方式	分阶段：感知→规划→执行	端到端：图像+语言→动作
泛化能力	需要精确建模	语言条件化，零样本泛化
适用场景	工业级重复操作	开放世界灵巧操作
本项目定位	V1 骨架验证	V2+ 演进方向

4.5 视觉模块（VisionModule）

- **ObjectDetectionCapability**：基于 RGB-D 图像进行物体检测（2D bbox + 类别）
- **PoseEstimationCapability**：结合 RGB-D 数据估计目标物体的 6DoF 位姿
- **VisualServoCapability**：实时视觉反馈驱动操作臂微调，与 ManipulationModule 闭环协作

4.6 搜索模块（SearchModule · SubAgent）

ObjectSearchCapability 内部调用 ModelProvider 推理目标物体最可能的位置（基于 SceneGraph），调用 NavigationCapability 前往候选位置，到达后通过 VLM 确认。

五、数据模型设计

5.1 3D 场景图数据结构

场景图采用层级化语义环境表征，参考 SayPlan 的 3DSG 方案：

- **SceneNode**：场景图节点，包含 node_id、name、node_type（Building/Floor/Room/Object）、position、properties
- **SceneEdge**：场景图边，表示节点间关系（contains/on/in/near/connected_to）
- **SceneGraph**：完整场景图，支持子图提取（get_subgraph）、诱导子图（get_induced_subgraph，用于 EmbodiedRAG）、JSON 序列化

5.2 任务与执行数据结构

- **TaskContext**：任务上下文，承载用户原始输入及环境信息
- **TaskResult**：任务解析结果，包含 intent、params、sub_tasks、confidence
- **Task**：可执行任务，包含 task_id、priority、status、retry_count
- **PlannedAction**：计划中的单个动作，绑定 Capability 名称
- **ExecutionPlan**：LLM 生成 + Validator 验证后的有序动作序列
- **ExecutionResult**：执行结果，沿管道回传
- **FeedbackEvent**：执行中反馈事件，类型包括 PROGRESS、BLOCKED、LOCALIZATION_LOST 等

5.3 ARIA 记忆数据结构

- **RobotState**：机器人实时状态（位姿、定位状态、电池、能力状态）
- **PlanningContext**：EmbodiedRAG 检索结果封装，包含精简子图、机器人状态、历史经验
- **ExecutionEpisode**：一次任务执行经历（情景记忆单元），包含任务描述、计划摘要、结果、环境快照、向量嵌入

VectorStore 选型：骨架阶段使用 ChromaDB（轻量级、嵌入式），接口层抽象为 VectorStore 协议，后续可切换至 FAISS 或 Milvus。

六、错误处理与降级策略

6.1 错误分类与处理

错误类型	处理策略
LLM API 失败	指数退避重试（最多3次），耗尽后返回错误
意图解析失败	返回错误，提示用户重新输入
Capability 执行失败	按配置重试，失败后返回错误
执行中路径阻塞	SceneGraph 更新 → LLM 重规划
执行中定位丢失	RobotState 更新 → LLM 评估并暂停或重定位
场景图构建失败	回退到已有场景图，降级运行
LLM 规划验证失败	携带反馈让 LLM 重新生成（最多 N 轮）

6.2 错误传播原则

1. 所有错误最终封装为 ExecutionResult 回传给用户
2. 单个 Capability 异常不影响其他 Capability 和 Agent 核心运行
3. 系统不产生未处理异常，任何阶段的错误都有明确的降级路径
4. 错误信息使用中文，对用户友好

七、测试策略

系统采用单元测试 + 属性测试的双轨方法。

- **属性测试**：使用 Hypothesis 库，每个属性测试最少运行 100 次迭代，覆盖 Registry round-trip、优先级队列排序、状态机合法转换、JSON 序列化 round-trip、LocationService round-trip、错误传播等核心正确性属性
- **单元测试**：使用 pytest 框架，聚焦具体示例、边界情况和错误条件，覆盖接口结构验证、Demo Capability 实现验证、配置文件结构验证、CLI 行为、端到端管道集成测试

八、骨架阶段实现策略

阶段	策略	目的
Step 1：管道验证	SceneGraphBuilder 从 JSON/YAML 加载预定义场景图（mock）	跑通 Builder → WSM → Planner 数据管道
Step 2：真实感知	MapAnalyzer 接入 SLAM 栅格地图，SceneAnalyzer 调 VLM API	验证 SLAM + VLM 融合构建场景图